



1. 二叉树的遍历

遍历

按某条搜索路径巡访树中每个节点，使得每个节点均被访问一次，而且仅被访问一次

遍历的本质就是对一个非线性结构进行线性化操作
使除第一个和最后一个节点外的所有节点在这些线性序列中有且仅有一个直接前驱和直接后继



1. 二叉树的遍历

①先序遍历

```
void PreOrder(BiTree T)
{
    if(T != NULL)
    {
        visit(T); //访问根节点
        PreOrder(T->lchild); //递归遍历左子树
        PreOrder(T->rchild); //递归遍历右子树
    }
}
```

遍历过程中每个结点都会经过3次

第1次经过时访问并输出它

然后开始遍历左子树

第2次经过是因为它的左子树已遍历完毕

第3次经过是因为它的右子树已遍历完毕



1. 二叉树的遍历

①先序遍历

```
void PreOrder(BiTree T)
{
    if(T != NULL)
    {
        visit(T); //访问根节点
        PreOrder(T->lchild); //递归遍历左子树
        PreOrder(T->rchild); //递归遍历右子树
    }
}
```

代码执行完毕

遍历过程中每个结点都会经过3次

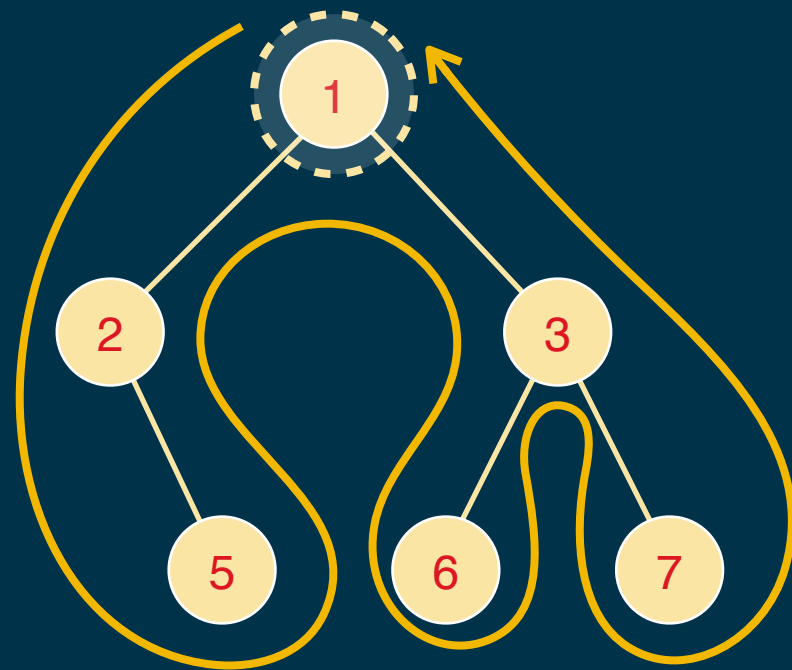
第1次经过时访问并输出它

然后开始遍历左子树

第2次经过是因为它的左子树已遍历完毕

第3次经过是因为它的右子树已遍历完毕

逻辑视角



遍历结果: 1 2 5 3 6 7



1. 二叉树的遍历

②中序遍历

```
void InOrder(BiTree T)
{
    if(T != NULL)
    {
        InOrder(T->lchild); //递归遍历左子树
        visit(T); //访问根节点
        InOrder(T->rchild); //递归遍历右子树
    }
}
```

遍历过程中每个结点都会经过3次
第1次经过是为了能访问到它的左子树
第2次经过是因为左子树已遍历完毕
此时访问并输出它
然后开始遍历右子树
第3次经过是因为它的右子树已遍历完毕



1. 二叉树的遍历

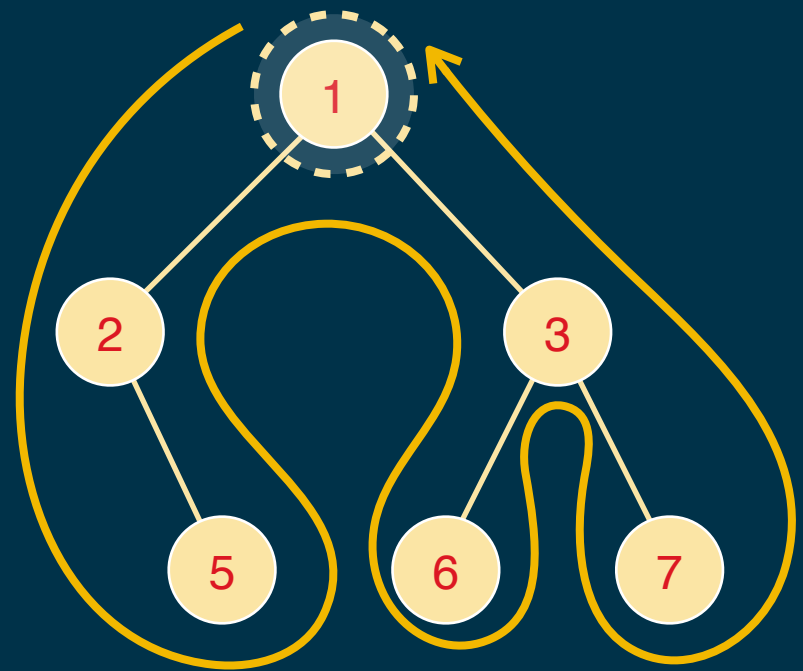
②中序遍历

```
void InOrder(BiTree T)
{
    if(T != NULL)
    {
        InOrder(T->lchild); //递归遍历左子树
        visit(T); //访问根节点
        InOrder(T->rchild); //递归遍历右子树
    }
}
```

代码执行完毕

遍历过程中每个结点都会经过3次
第1次经过是为了能访问到它的左子树
第2次经过是因为左子树已遍历完毕
此时访问并输出它
然后开始遍历右子树
第3次经过是因为它的右子树已遍历完毕

逻辑视角



遍历结果: 2 5 1 6 3 7



1. 二叉树的遍历

③后序遍历

```
void PostOrder(BiTree T)
{
    if(T != NULL)
    {
        PostOrder(T->lchild); //递归遍历左子树
        PostOrder(T->rchild); //递归遍历右子树
        visit(T); //访问根节点
    }
}
```

遍历过程中每个结点都会经过3次
第1次经过是为了能访问到它的左子树
第2次经过是为了能访问到它的右子树
第3次经过是因为左右子树已遍历完毕
此时访问并输出它



1. 二叉树的遍历

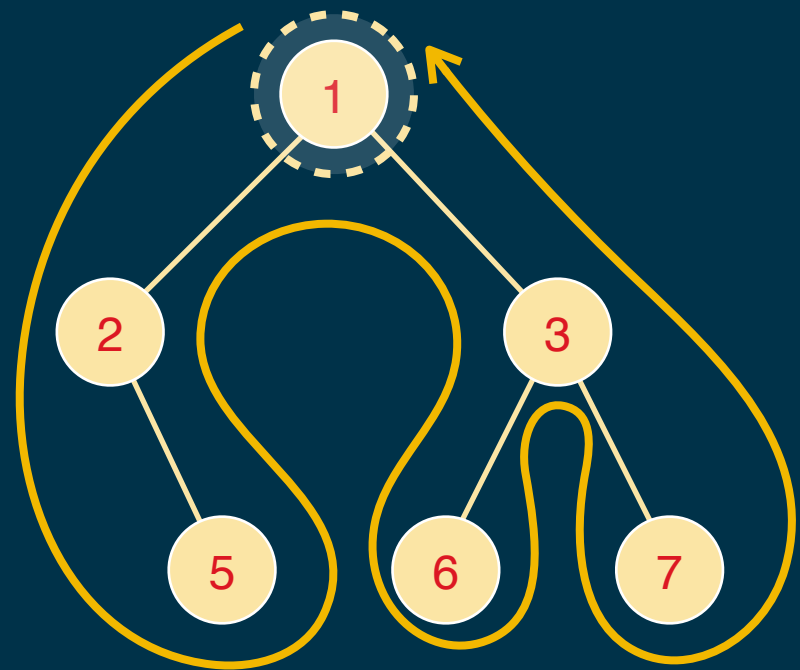
③后序遍历

```
void PostOrder(BiTree T)
{
    if(T != NULL)
    {
        PostOrder(T->lchild); //递归遍历左子树
        PostOrder(T->rchild); //递归遍历右子树
        visit(T); //访问根节点
    }
}
```

代码执行完毕

遍历过程中每个结点都会经过3次
第1次经过是为了能访问到它的左子树
第2次经过是为了能访问到它的右子树
第3次经过是因为左右子树已遍历完毕
此时访问并输出它

逻辑视角



遍历结果: 5 2 6 7 3 1



1. 二叉树的遍历

④层序遍历

```
void LevelOrder(BiTree T)
{
    InitQueue(Q); //初始化辅助队列
    BiTree p;
    EnQueue(Q,T); //根节点入队
    while(!IsEmpty(Q)){ //队列不空则循环
        DeQueue(Q,p); //队头节点出队
        visit(p); //访问出队节点
        if(p->lchild != NULL)
            EnQueue(Q,p->lchild); //若左孩子不空，则左孩子入队
        if(p->rchild != NULL)
            EnQueue(Q,p->rchild); //若右孩子不空，则右孩子入队
    }
}
```

当队列不空时就出队队头元素并访问输出
然后将队头元素所有的邻接点（左右孩子）入队
不断重复这个过程，直到队列为空



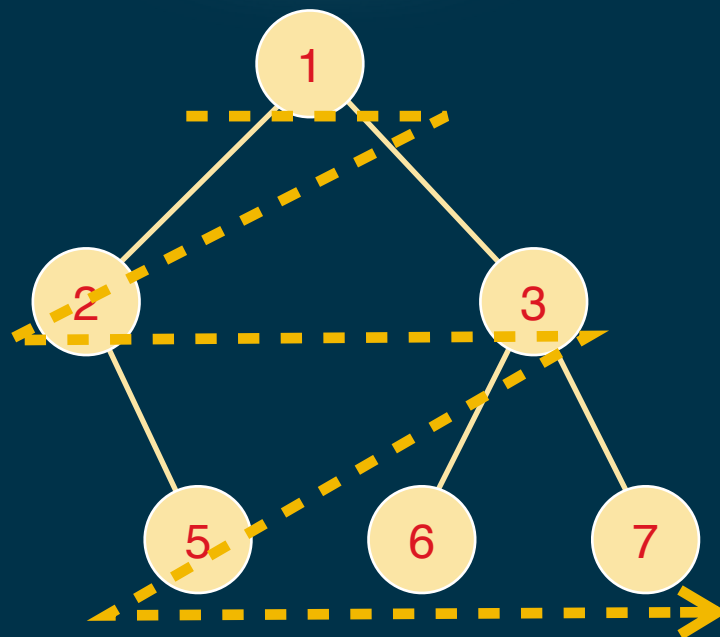
1. 二叉树的遍历

④层序遍历

```
void LevelOrder(BiTree T)
{
    InitQueue(Q); //初始化辅助队列
    BiTree p;
    EnQueue(Q,T); //根节点入队
    while(!IsEmpty(Q)){ //队列不空则循环
        DeQueue(Q,p); //队头节点出队
        visit(p); //访问出队节点
        if(p->lchild != NULL)
            EnQueue(Q,p->lchild); //若左孩子不空，则左孩子入队
        if(p->rchild != NULL)
            EnQueue(Q,p->rchild); //若右孩子不空，则右孩子入队
    }
}
```

当队列不空时就出队队头元素并访问输出
然后将队头元素所有的邻接点（左右孩子）入队
不断重复这个过程，直到队列为空

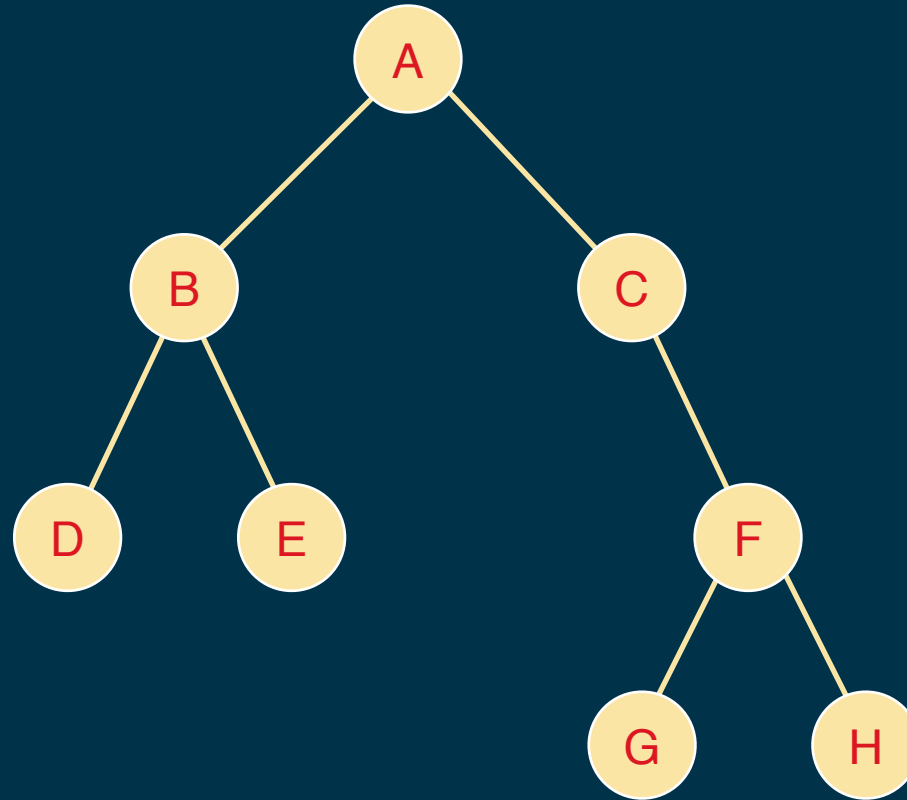
逻辑视角



层序遍历序列



1. 二叉树的遍历





2. 由遍历序列构造二叉树

只知前序遍历序列，中序遍历序列，后序遍历序列中的**任意一个**的时候，无法唯一确定一棵二叉树的

已知中序遍历序列+其他三种遍历序列的任意一种，即可确定唯一的一棵二叉树



2. 由遍历序列构造二叉树

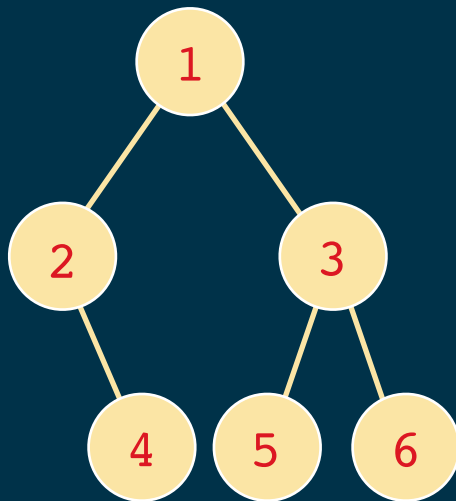
只知前序遍历序列，中序遍历序列，后序遍历序列中的**任意一个**的时候，无法唯一确定一棵二叉树的
已知**中序遍历序列+其他三种遍历序列的任意一种**，即可确定唯一的一棵二叉树

已知二叉树先序遍历序列为1, 2, 4, 3, 5, 6
求该二叉树

中序遍历序列为2, 4, 1, 5, 3, 6

先序遍历：根 左 右

中序遍历：左 根 右

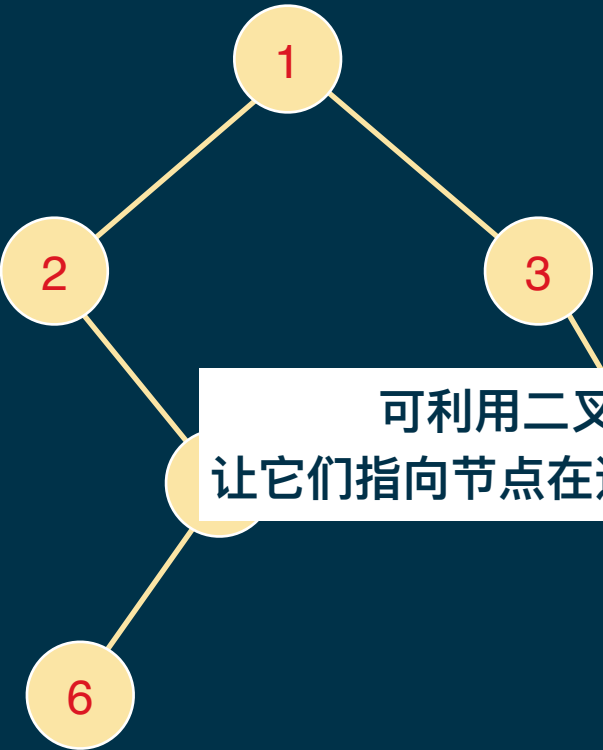




3. 线索二叉树

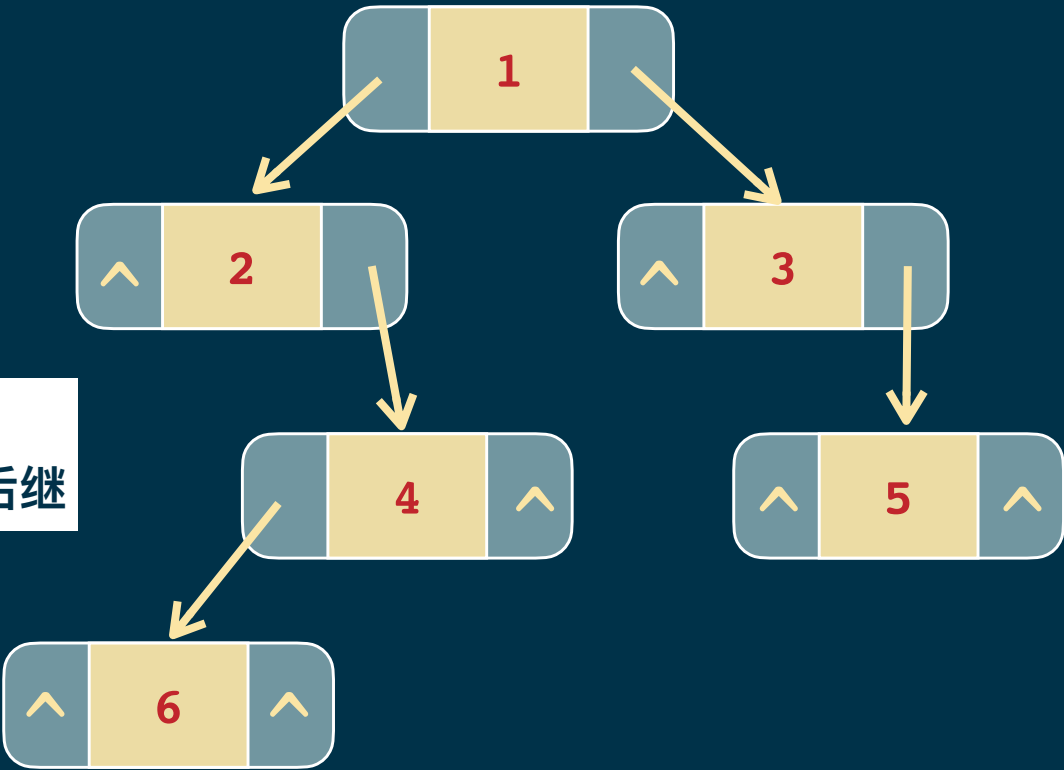
①线索二叉树的基本概念

逻辑视角



可利用二叉链表中的空指针
让它们指向节点在遍历序列中的前驱或后继

内存视角





3. 线索二叉树

①线索二叉树的基本概念



lTag=0	lchild指示结点的左孩子
lTag=1	lchild指示结点的前驱
rTag=0	rchild指示结点的右孩子
rTag=1	rchild指示结点的后继



3. 线索二叉树

① 线索二叉树的基本概念

```
typedef struct ThreadNode{
    ElemType data; //数据元素
    struct ThreadNode *lchild, *rchild; //左右孩子指针
    int ltag, rtag; //左右线索标志
}ThreadNode, *ThreadTree;
```



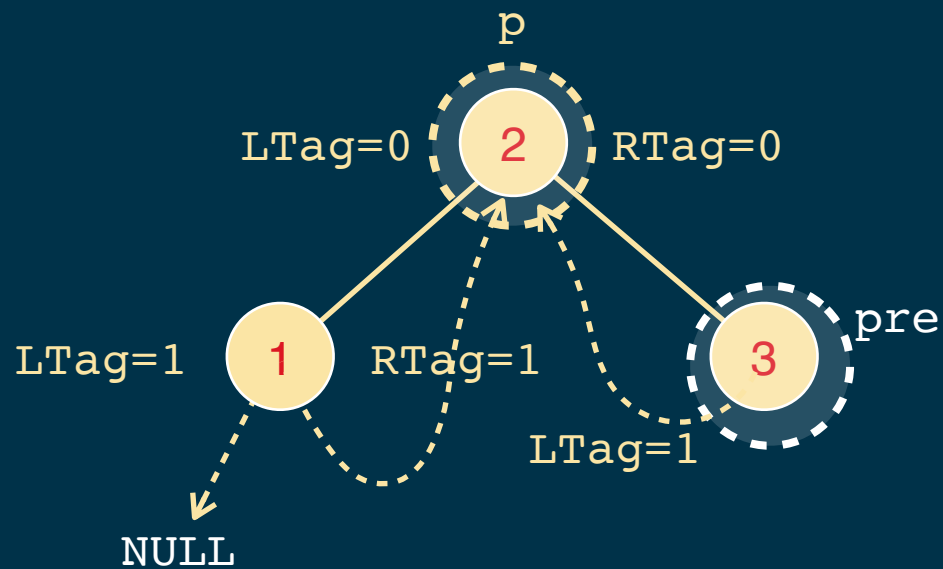
3. 线索二叉树

②构造线索二叉树（线索化）

```
void InThreading(BiThrTree p)
{
    //pre 是全局变量，初始化时其右孩子指针为空，便于在树的最左点
    //开始建线索
    if(p)
    {
        InThreading(p->lchild);    //左子树递归线索化
        if(!p->lchild)             //p的左孩子为空
        {
            p->LTag=1;             //给p加上左线索
            p->lchild=pre;         //p的左孩子指针指向pre（前驱）
        }
        else p->LTag=0;
        if(!pre->rchild)           //pre的右孩子为空
        {
            pre->RTag=1;           //给pre加上右线索
            pre->rchild=p;         //pre的右孩子指针指向p（后继）
        }
        else pre->RTag=0;
        pre=p;                    //保持pre指向p
        InThreading(p->rchild);    //右子树递归线索化
    }
}
```

代码运行完毕

逻辑视角





3. 线索二叉树

③遍历线索二叉树

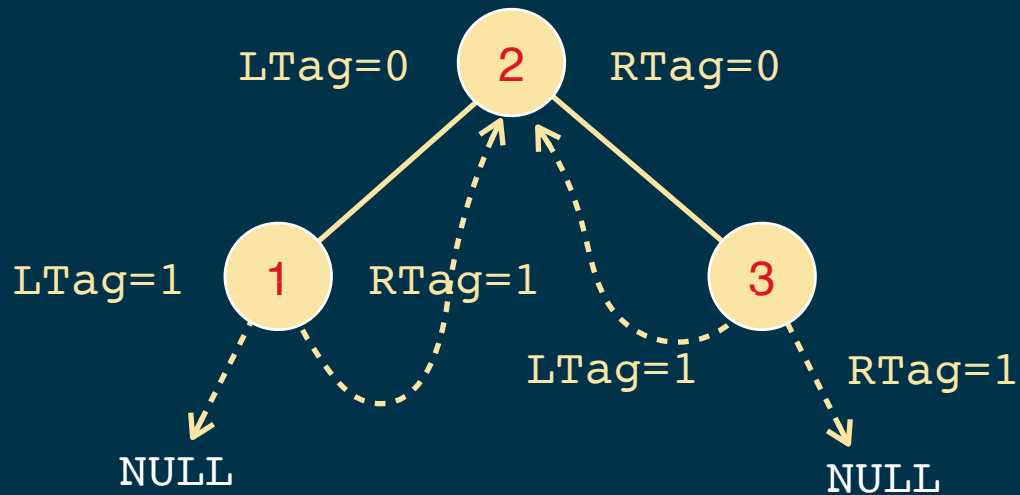
中序线索二叉树的结点中已隐含前驱和后继信息，因此遍历时无须借助栈或递归

```
ThreadNode *Firstnode (ThreadNode *p) {
    while (p->ltag==0) p = p->lchild; //沿左孩子链走到最左下结点
    return p;
}
```

```
ThreadNode *Nextnode (ThreadNode *p) {
    if (p->rtag==0)
        return Firstnode (p->rchild); //右子树中最左下结点
    else return p->rchild; //若rtag==1 则直接返回后继线索
}
```

```
void Inorder (ThreadNode *T) {
    for (ThreadNode *p=Firstnode (T); p!=NULL; p=Nextnode (p))
        visit (p) ;
}
```

逻辑视角



祝你今天特别特别开心！